

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Deepan Kumar P	Reg. No.:	192312498
Section / Batch:	BE ECE 4 th year	Date:	25/08/2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time. • Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Design an I/O communication mechanism for a smart agriculture system with the following constraints:
 - 50 field sensors transmit readings at different intervals.
 - Soil-moisture alerts must be processed within 20 μ s.
 - Use interrupt-driven I/O for critical events.
 - Use DMA for periodic bulk sensor-data transfer.
2. Construct an I/O subsystem for a video surveillance system with the following constraints:
 - 8 cameras continuously generate video streams.
 - The aggregate data rate is 1.5 GB/s.
 - Minimize CPU involvement during video transfer.
 - Use DMA with buffer management to prevent data loss.
3. Design a peripheral communication architecture for an industrial automation system with the following constraints:
 - 12 peripherals operate at different data rates.
 - High-priority control devices require deterministic response.
 - Low-priority devices must not monopolize the communication bus.
 - Implement priority-based bus arbitration.
4. Construct a data transfer mechanism for a real-time audio processing system with the following constraints:
 - Audio data must be transferred continuously at 200 MB/s.
 - Transfer interruptions must be minimized.
 - Use double buffering for continuous data processing.
 - Generate an interrupt only when a buffer transfer is completed.
5. Design an interrupt management system for a multicore embedded processor with the following constraints:

- Support multiple interrupt sources simultaneously.
- Distribute interrupts across processor cores to balance the workload.
- Critical interrupts must have higher priority than routine interrupts.
- Prevent excessive interrupt overhead during high system activity.

Code of Conduct

I Deepan Kumar P (192312498) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	

4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Q1. I/O Communication Mechanism for Smart Agriculture

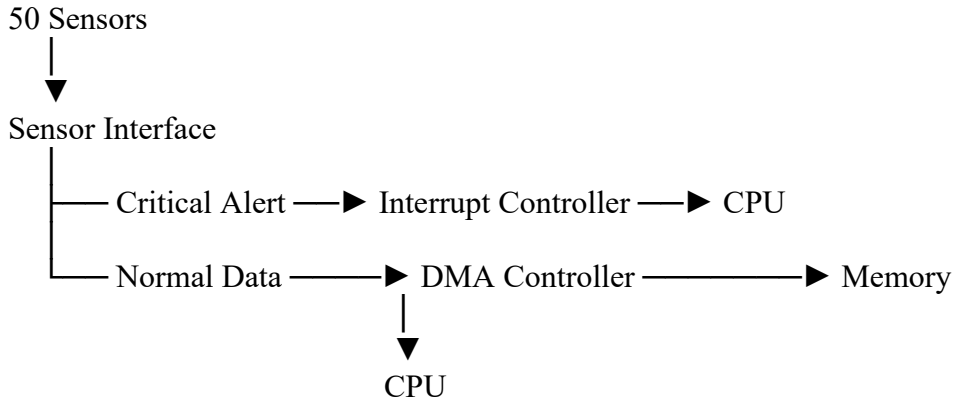
1. Problem Understanding

The smart agriculture system contains **50 field sensors** transmitting readings at different intervals. Soil-moisture alerts are critical and must be processed within **20 μ s**. Therefore, the system requires fast interrupt handling for critical events and efficient bulk transfer for normal sensor data.

2. Constraints

- 50 sensors with different transmission intervals.
- Soil-moisture alert response $\leq 20 \mu$ s.
- Interrupt-driven I/O for critical events.
- DMA for periodic bulk data transfer.
- CPU utilization must be minimized.

3. Proposed Architecture



4. Application of Concepts

Interrupt-driven I/O:

When a soil-moisture value crosses the critical threshold, the sensor generates an interrupt. The interrupt controller immediately notifies the CPU.

DMA:

Periodic readings from all 50 sensors are collected into a buffer and transferred directly to memory using DMA. The CPU is interrupted only after a DMA transfer is completed.

Priority:

Soil-moisture alerts are assigned the highest priority so that they meet the 20 μ s response requirement.

5. Justification

Interrupt-driven I/O is selected for critical events because waiting for normal polling could increase response time. DMA is used for periodic sensor data because transferring every reading through the CPU would increase CPU overhead.

Final decision: Use priority-based interrupt handling for alerts + DMA for bulk sensor data. This provides fast response while keeping CPU utilization low.

Q2. I/O Subsystem for Video Surveillance

1. Problem Understanding

The system has **8 continuously operating cameras** producing an aggregate data rate of **1.5 GB/s**. CPU involvement must be minimized, and DMA with buffer management must prevent video data loss.

2. Constraints

- 8 continuous video streams.
- Aggregate data rate = 1.5 GB/s.
- Minimum CPU involvement.
- DMA-based transfer.
- Buffer management to prevent data loss.

3.Buffer Management

A circular or multiple-buffer system can be used:

Camera → DMA → Buffer A → Processing

Buffer B → Processing

Buffer C → Processing

While the processor processes one buffer, DMA fills another buffer.

4. Application of Concepts

DMA transfers video data directly from the camera interface to memory without continuously involving the CPU.

Buffer management prevents newly arriving video data from overwriting data that is still being processed.

5. Justification

DMA is the best choice because the video stream has a very high continuous data rate. CPU-controlled transfers would create excessive overhead. Multiple buffers provide continuous data flow and reduce the possibility of frame loss.

Final decision: Use **multi-buffer DMA with high-speed memory and efficient buffer management.**

Q3.Design a Peripheral Communication Architecture for an Industrial Automation System

1. Problem Understanding

The industrial automation system contains 12 peripherals operating at different data rates. High-priority control devices require deterministic response, while low-priority devices must also receive communication access. Therefore, a priority-based bus arbitration mechanism is required.

2. Constraints

1. 12 peripherals operate at different data rates.
2. High-priority control devices require deterministic response.
3. Low-priority devices must not monopolize the bus.
4. Priority-based bus arbitration must be implemented.
5. Bus access should be efficient and fair.

3. Proposed Solution

A central bus arbiter can be used to control access to the communication bus.

Each peripheral sends a bus request when it needs to communicate. The bus arbiter evaluates the requests and grants access according to the assigned priority.

High-priority control devices receive bus access before low-priority devices.

4. Priority Assignment

A suitable priority arrangement is:

1. Emergency and critical control devices – Highest priority
2. Real-time control and sensors – High priority
3. Normal controllers – Medium priority
4. Monitoring devices – Low priority

5. Preventing Starvation

A fixed-priority system may cause low-priority devices to wait for a long time. To prevent this, an aging mechanism can be used.

When a low-priority device waits for a long period, its effective priority is gradually increased. This allows it to eventually receive bus access.

6. Justification

Priority-based arbitration ensures that critical control devices receive fast and predictable bus access.

Aging prevents low-priority devices from being permanently blocked.

Q4. Construct a Data Transfer Mechanism for a Real-Time Audio Processing System

1. Problem Understanding

The real-time audio system must continuously transfer audio data at 200 MB/s. Transfer interruptions must be minimized. Therefore, the system requires continuous data movement and efficient buffer management.

2. Constraints

1. Continuous data transfer rate is 200 MB/s.
2. Transfer interruptions must be minimized.
3. Double buffering must be used.
4. An interrupt must be generated only when a buffer transfer is completed.

3. Proposed Solution

DMA is used to transfer audio data directly between the audio interface and memory.

Two buffers, called Buffer A and Buffer B, are used.

While DMA transfers new audio data into Buffer A, the processor processes the data already available in Buffer B. After Buffer A is completely filled, DMA switches to Buffer B and the processor starts processing Buffer A.

This process continues repeatedly.

4. Transfer Analysis

Required data transfer rate:

200 MB/s

For a buffer of size B:

$$T = B / 200 \text{ MB/s}$$

For example, if the buffer size is 1 MB:

$$T = 1 \text{ MB} / 200 \text{ MB/s}$$

$$T = 0.005 \text{ s}$$

$$T = 5 \text{ ms}$$

Therefore, a 1 MB buffer would complete approximately every 5 ms.

5. Interrupt Handling

An interrupt is generated only after the DMA completes transferring one buffer.

This reduces the number of interrupts compared with generating an interrupt for every individual audio sample or data block.

6. Justification

DMA provides continuous high-speed data transfer without continuously involving the CPU. Double buffering allows data transfer and processing to occur simultaneously.

Therefore, **DMA with double buffering and buffer-completion interrupts** is the most suitable solution for continuous real-time audio processing.

Q5. Design an Interrupt Management System for a Multicore Embedded Processor

1. Problem Understanding

The multicore embedded processor must handle multiple interrupt sources simultaneously. Interrupts need to be distributed among processor cores to balance the workload. Critical interrupts must have higher priority, while excessive interrupt overhead must be avoided.

2. Constraints

1. Multiple interrupt sources must be supported.
2. Interrupts must be distributed among processor cores.
3. Critical interrupts must have higher priority.
4. Processor workload must be balanced.
5. Excessive interrupt overhead must be minimized.

3. Proposed Solution

A multicore interrupt controller can be used to receive interrupts from different devices and route them to suitable processor cores.

Critical interrupts are assigned higher priority and are delivered immediately to an available processor core.

Routine interrupts can be distributed among the remaining cores according to their workload.

4. Priority Scheme

A suitable priority order is:

1. Emergency and critical interrupts – Highest priority
2. Real-time control interrupts – High priority
3. Communication interrupts – Medium priority
4. Routine monitoring interrupts – Low priority

5. Load Balancing

Interrupts should not always be directed to a single processor core.

For example, critical control interrupts can be handled by one core, communication interrupts by another core, sensor interrupts by another core, and routine interrupts by another available core.

The assignment can also be dynamically changed according to processor workload.

6. Reducing Interrupt Overhead

The following methods can be used:

1. Interrupt prioritization for critical events.
2. Interrupt masking for unnecessary routine events.
3. Interrupt coalescing for frequent interrupts.
4. DMA for high-volume data transfers.
5. Load balancing across processor cores.

Interrupt coalescing allows several related events to be handled using a single interrupt instead of generating an interrupt for every individual event.

7. Justification

Priority-based interrupt handling ensures that critical events receive immediate attention. Distributing interrupts among multiple cores prevents a single core from becoming overloaded. DMA and interrupt coalescing further reduce CPU interrupt overhead.

Therefore, the best solution is a **multicore interrupt controller using priority-based routing, load balancing, interrupt coalescing, and DMA where appropriate.**